

Mastering ES6 in Lightning Web Components (LWC)

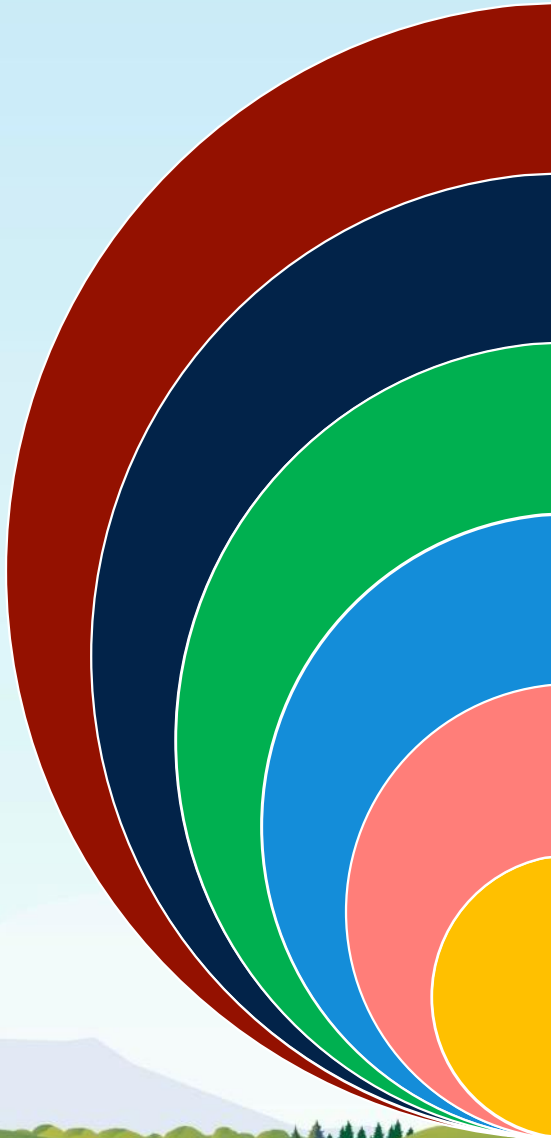
Introduction

Sheetal Chougule



Agenda

salesforce



01

Understand and use ES6 features in LWC.

02

Apply let, const, arrow functions, spread operator.

03

Use ES6 modules and import/export in LWC.

04

Work with LWC-specific decorators like @api, @track, @wire.

05

Understand DOM manipulation using querySelector.

06

Implement content distribution using slots.



Native JS and ES6



What are web components?

Faster, more responsive and compatible with most devices and browsers

- Custom Elements
 - Define your own custom HTML tags like `<hello-world>`
- Shadow DOM
 - Create scoped sub-trees in DOM separating the component level styling
 - Used for encapsulation
- HTML templates
 - The elements `<template>` and `<slot>` allow to create reusable markup for custom elements
 - HTML templates are rendered at runtime by JavaScript



Benefits of web components



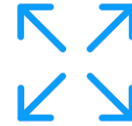
Reusability



Interoperability



Encapsulation



Extensibility



Composability



Native JS and ES6 (ECMAScript2015)

- **“Native”** - features or functionalities that are implemented directly in the JavaScript engine, without requiring additional libraries or frameworks. These are the built-in capabilities provided by the language itself.
- **“ES6” (ECMAScript 2015)** - refers to the sixth edition of the ECMAScript standard, which introduced significant enhancements and new features to the JavaScript language. These features are part of the ECMAScript standard and are supported by modern JavaScript engines.

Key Differences Between CommonJS and ES Modules		
Feature	CommonJS (CJS)	ES Modules (ESM)
Syntax	<code>require()</code> and <code>module.exports</code>	<code>import</code> and <code>export</code>
Execution	Synchronous	Asynchronous
Dynamic Imports	Supported	Requires <code>import()</code>
Tree Shaking	Not supported	Supported
File Extensions	<code>.js</code> or <code>.cjs</code>	<code>.mjs</code> or <code>.js</code>
Node.js Default	Default module system	Requires explicit setup

ES6 Features

salesforce



Arrow Functions

01

A concise syntax for writing function expressions, with implicit return and lexical this binding



let & const

02

Block-scoped variable declarations, providing better control over variable scope and immutability with const.



Template Literals

03

A more expressive way to create strings using backticks (`) for string interpolation and multiline strings.



Destructuring Assignment

04

Extracting values from arrays or objects into individual variables using a shorthand syntax.



Spread & Rest

05

Spread (...) for expanding arrays/iterables into individual elements, and Rest (...) for gathering remaining parameters into an array.

ES6 Features

salesforce



Classes

06

Syntactic sugar for prototype-based inheritance, providing a more familiar syntax for defining object-oriented classes.



Promises

07

A built-in mechanism for handling asynchronous operations, simplifying asynchronous code and error handling with `then()` and `catch()` methods.



Modules

08

A standardized way to organize and structure JavaScript code into reusable modules, improving code maintainability and scalability.



Async/Await

09

Syntactic sugar for working with asynchronous code, making it easier to write and understand asynchronous functions using `async` and `await` keywords.



Map, Set, WeakMap, WeakSet

10

New data structures for storing unique values and key-value pairs, providing alternatives to traditional arrays and objects.



Understanding ES6 in LWC



Javascript Standards for LWC

- Every component must have a JavaScript file. JavaScript files in Lightning web components are ECMAScript 6 (ES6) modules
- LWC uses ES6(2015), ES7(2016), ES8(2017), ES9(2018), ES10(2019), and ES11(2020) versions of JavaScript, which are the modern version of JavaScript.
- Important JS standards and changes you need to be aware before starting with Lightning Web Components:
 - Decorators
 - DispatchEvent and CustomEvent
 - QuerySelectors
 - Import/Export
 - Spread Notation
 - Slots
 - Promises
 - String Interpolation + Let/Const



ES6 Modules (import/export)

- An ES6 module is a JavaScript file that exports variables or functions that other modules can use.
- The Primary utility of a module is to help with code reusability and code structuring.
- Modules scoped with @salesforce add functionality to Lightning web components at runtime.
- A module is a file that encapsulates related code, can include functions, classes, or variables
- Import and export statements are used to use and create modules.
- **Exporting** - make parts of a module available for use in other modules. You can export functions, classes, variables, and more. There are two types of exports: named exports and default exports.
- **Importing** - bring in exports from other modules to use in your current module. You can import named exports using curly braces and default exports without curly braces.

```
// mathUtils.js
export const square = num => num * num;

// useUtils.js
import { square } from './mathUtils';
console.log(square(4)); // 16
```

let, const & Arrow Functions

- **let**: It can be updated but cannot re-declared. Support function and block level scope. It support global level scope but does not create property on global object
- **const**: It cannot be updated also cannot re-declared. Support function and block level scope. It support global level scope but does not create property on global object.
- Arrow Function is concise way of writing Javascript functions in shorter way.

```
1 let arrArrow = [1,2,3]
2 let arrValue = arrArrow.map((currentItem)=>{
3     console.log(arrArrow);
4     return currentItem*10
5 })
6 //let arrValue = arrArrow.map((currentItem)=>currentItem*10)
7 console.log('Arrow Function:- '+arrValue); // Output :- 10,20,30
```

```
// greeting.js
let name = 'John';
const greeting = `Hello, ${name}!`;

const getUppercase = msg => msg.toUpperCase();
console.log(getUppercase(greeting));
```



Spread Operator

- **The Spread operator** allows an iterable to expand in places where 0+ arguments are expected.
- It is mostly used in the variable array where there is more than 1 value is expected. It allows us the privilege to obtain a list of parameters from an array.
- allows you to expand an iterable (like an array or object) into individual elements or properties.
- useful for merging objects, copying arrays, and passing multiple properties to child components. Essentially, it simplifies code by avoiding verbose loops or repetitive assignments.

```
const baseCourses = ['LWC', 'Apex'];
const extraCourses = ['Flows', 'SOQL'];
const allCourses = [...baseCourses, ...extraCourses]; // Array merge

const contact = { name: 'Ram', city: 'Pune' };
const updatedContact = { ...contact, city: 'Mumbai' }; // Override
```

Decorators - @api, @track, @wire

- Decorators are special functions used to add functionality to properties and methods. They modify the behavior of the decorated element, making it reactive, public, or connected to Salesforce data.
- Three primary decorators exist: @api, @track, and @wire.
 - @api:** Exposes properties and methods as public, making them accessible from parent components. Automatically makes these public properties reactive, meaning changes trigger component re-rendering.
 - @track:** Declares private properties within a component that are reactive. Used for observing changes in the values of objects and arrays, ensuring the component re-renders when these changes occur.
 - @wire:** Connects components to Salesforce data or services, commonly used with Apex methods or wire adapters. Provides a reactive way to fetch and interact with Salesforce data. When the data changes, the component automatically re-renders.

```
// childComp.js
import { LightningElement, api } from 'lwc';

export default class ChildComp extends LightningElement {
  @api title; // public property
}
```

```
// parentComp.html
<c-child-comp title="Welcome Trainees"></c-child-comp>
```

```
import { LightningElement, wire } from 'lwc';
import getAccounts from
  '@salesforce/apex/AccountController.getAccounts';

export default class AccountList extends
  LightningElement {
  @wire(getAccounts) accounts;
}
```

querySelector DOM Access

- Query selectors in Lightning Web Components (LWC) are methods used to select and manipulate DOM elements within a component's template. They allow you to access and interact with specific elements based on their CSS selectors.
- `querySelector` is used to select a single DOM element within the component's template.
- It is similar to JavaScript's standard `document.querySelector`, but it operates within the Shadow DOM, meaning it only accesses elements inside your LWC component's template.

```
// querySelectorDemo.html
<template>
  <div class="box" onclick=
{highlight}>Click Me</div>
</template>
```

```
// querySelectorDemo.js
export default class QuerySelectorDemo extends
LightningElement {
  highlight() {
    const div =
this.template.querySelector('.box');
    div.style.backgroundColor = 'yellow';
  }
}
```

Slots for Content Distribution

- slots are used to create flexible and reusable components by enabling content distribution.
- placeholders within a child component's template where a parent component can inject its own content; allowing for dynamic content injection and promotes component reusability across different contexts.
- A `<slot>` element within a child component's template acts as a designated area for content injection.
- A parent component can insert its own HTML markup into the child component's slots using the slot attribute.
- LWC supports both default (unnamed) slots, which accept any content passed to the component, and named slots, which allow for specific content targeting.

```
<!-- childComponent.html -->
<template>
  <p>Header: <slot name="header"></slot></p>
  <p>Content: <slot></slot></p>
  <p>Footer: <slot name="footer"></slot></p>
</template>
```

```
<!-- parentComponent.html -->
<template>
  <c-child-component>
    <span slot="header">This is the
header</span>
    <span>This is the main content</span>
    <div slot="footer">This is the footer</div>
  </c-child-component>
</template>
```



salesforce

References

- [Component library](#)
- [Playground to Test](#)
- [LWC Dev Guide](#)



Rhea Sy

Lead Developer, 21st Century Fox

Nithin Angri

Program Manager, AWS



thank you

Family Reunion

